

Object-Oriented Programming Using Java

Interface and Abstract Class

Contents

- Defining an Interface
- Implementing an Interface
- Abstract Methods and Classes

References:

<https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html>

<https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html>

Interface

- *Interface* is a reference type (kiểu dữ liệu tham khảo), similar to a class
- *Interface* can contain *only*
 - constants,
 - method signatures,
 - abstract methods,
 - default methods,
 - static methods,
 - and nested types

Defining an Interface

```
public interface IShape {  
    public static final double PI = 3.14;  
  
    public double calculateArea(); // method signatures  
    public abstract double calculatePerimeter(); // abstract methods  
  
    //All methods in an interface are implicitly public,  
    //you can omit the public modifier.  
  
    //Only default and static methods have implementations  
    static double sqr(double x){  
        return x*x;  
    }  
  
}
```

Implementing an Interface

```
34  class Rectangle implements InterfaceShape{
35      int length, width;
36      public Rectangle(int length, int width) {
37          this.length = length;
38          this.width = width;
39      }
40      @Override
41      public double calculateArea() {
42          return length*width;
43      }
44      @Override
45      public double calculatePerimeter() {
46          return (length+width)*2;
47      }
48  }
```

Implementing an Interface

```
57 class Circle implements InterfaceShape{
58     double radius ;
59
60     public Circle(double radius) {
61         this.radius = radius;
62     }
63
64     @Override
65     public double calculateArea() {
66         return InterfaceShape.sqr(radius)*InterfaceShape.PI;
67     }
68
69     @Override
70     public double calculatePerimeter() {
71         return 2*radius*InterfaceShape.PI;
72     }
73 }
```

Implementing an Interface

```
public static void main(String[] args) {  
    InterfaceShape arrShape[]=new InterfaceShape[5];  
    arrShape[0] = new Rectangle(20, 10);  
    arrShape[1] = new Circle(15);  
    arrShape[2] = new Rectangle(30, 20);  
    arrShape[3] = new Circle(25);  
    arrShape[4] = new Rectangle(90, 60);  
    for (InterfaceShape s : arrShape) {  
        System.out.println(s.calculateArea());  
    }  
}
```

Abstract Classes

- Lớp có khai báo *abstract* được gọi là lớp trừu tượng

```
class Person{  
    private String name;  
    private int age;  
  
    public void input() {  
        //input name and age  
    }  
  
    public void output() {  
        //output name and age  
    }  
}
```

```
abstract class Person{  
    private String name;  
    private int age;  
  
    public void input() {  
        //input name and age  
    }  
  
    public void output() {  
        //output name and age  
    }  
}
```

Ví dụ

```
abstract class GraphicObject {  
    int x, y; // fields  
    void moveTo(int newX, int newY){ // non-abstract methods  
        x = newX;  
        y = newY;  
    }  
    abstract void draw(); // abstract methods  
    abstract void resize();  
}  
  
// Implement all abstract methods  
class Rectangle extends GraphicObject {  
    @Override  
    void draw() { //code here }  
    void resize() { //code here }  
}
```

Lưu ý 01

1. Không thể tạo đối tượng từ lớp Abstract

Lưu ý 02

- Nếu bên trong của 1 lớp có phương thức *abstract* thì lớp đó bắt buộc khai báo là *abstract*

```
57
58
59 class GraphicObject {
60     int x, y; // fields
61     void moveTo(int newX, int newY){ // non-abstract methods
62         x = newX;
63         y = newY;
64     }
65     abstract void draw(); // abstract methods
66     abstract void resize();
67 }
```

Lưu ý 03

- Nếu
 - lớp con kế thừa từ lớp cha là abstract và
 - lớp con không hiện thực hết tất cả phương thức abstract của lớp cha
- Thì lớp con bắt buộc khai báo là *abstract*.

```
57 abstract class GraphicObject {  
59     int x, y; // fields  
60     void moveTo(int newX, int newY){ // non-abstract methods  
61         x = newX;  
62         y = newY;  
63     }  
64     abstract void draw(); // abstract methods  
65     abstract void resize();  
66 }  
67  
68 // Implement all abstract methods  
69 abstract class Rectangle extends GraphicObject {  
70     @Override  
71     void draw() {  
72         //code here  
73     }  
74 }
```

Abstract Class Implements an Interface



class X must
be abstract

```
abstract class X implements Y {  
    // implements all but one method of Y  
}  
  
class XX extends X {  
    // implements the remaining method in Y  
}
```

Abstract Classes

Compared to Interfaces

- **Can not instantiate** abstract class and interface
- With interfaces, all fields are automatically *public*, *static*, and *final*, all methods are *public*.
- A class can extend only one class, and **implement any number of interfaces.**

Test 1

```
public interface Document {  
    String NAME = "Document1";  
    String getExtension();  
  
    public static void main(String[] args) {  
        Document doc1 = new Document();  
        System.out.println(doc1.NAME);  
    }  
}
```

a) Document1

b) Compile-time error

c) Run-time error

d) Nothing

Document is
abstract; cannot be
instantiated

Test 2

```
public interface Document {
    String NAME = "Document1";
    void output(){
        System.out.println(NAME);
    }
    String getExtension();

    public static void main(String[] args) {
        Document doc1 = new JWord();
        doc1.output();
    }
}

class JWord implements Document{
    @Override
    public String getExtension() {
        return "jdoc";
    }
}
```

a) Document1

b) Compile-time error

c) Run-time error

d) Nothing

interface abstract
methods cannot
have body

Test 3

```
public interface Document {
    String NAME = "Document1";
    void output();
    String getExtension();
    public static void main(String[] args) {
        Document doc1 = new JWord();
        doc1.output();
    }
}

class JWord implements Document{
    String name; double size;
    void output() {
        System.out.println(Document.NAME + size);
    }
    String getExtension() {
        return "jdoc";
    }
}
```

a) Document10

b) Compile-time error

c) Run-time error

d) Document10.0

String getExtension()
attempting to assign
weaker access privileges;
was public

Test 4

```
abstract class Document {  
    String name;  
    double size;  
    void input(){  
        name = "Test";  
        size = 12.5;  
    }  
    void output() {  
        System.out.println(name + size);  
    }  
    public static void main(String[] args) {  
        Document doc1 = new Document();  
        doc1.input();  
        doc1.output();  
    }  
}
```

a) Test12.5

b) Compile-time error

c) Run-time error

d) null

Document is
abstract; cannot be
instantiated

Test 5

```
public interface Document {
    void create();
    void save();

    public static void main(String[] args) {
        Document doc1 = new Word();
        doc1.save();
    }
}

abstract class AWord implements Document{
    String name;
    double size;
    public void create(){
        name = "AWord1";
        size=0;
    }
}

class JWord extends AWord{
    public void save() {
        System.out.println("Save "+ name +" successfully!");
    }
}
```

a) Save null successfully!

b) Compile-time error

c) Run-time error

d) Save DWord1 successfully!

Test 6

```
public abstract class Document {
    abstract protected void create();
    abstract void save();

    public static void main(String[] args) {
        Document doc1 = new JWord();
        doc1.save();
    }
}

abstract class AWord extends Document{
    String name;
    double size;
    protected void create(){
        name = "AWord1";
        size=0;
    }
}

class JWord extends AWord{
    void save() {
        System.out.println("Save "+ name +" successfully!");
    }
}
```

a) Save null successfully!

b) Compile-time error

c) Run-time error

d) Save DWord1 successfully!